Atlas Industries: Enterprise Assistant — Agentic Internal Knowledge Chatbot

Project Description

Atlas Industries is an agentic internal knowledge chatbot built for a fictional mid-sized company (~500 employees). Employees ask plain-language questions and receive clear, cited answers drawn exclusively from official company documents — without searching through scattered PDFs, Word files, or intranet pages.

The system combines an **agentic RAG pipeline** over 30 internal documents (HR, IT Support, Finance & Reimbursement), a **LangGraph workflow** that routes each question to the correct domain, **session memory** so follow-up questions resolve naturally, **cited sources** on every answer, and a **DeepEval evaluation suite** over 10 gold test cases.

The Corpus

30 documents spread across three domains and three file formats:

Domain	Count	Formats
HR	10	`.md`, `.pdf`, `.docx`
IT Support	10	`.md`, `.pdf`, `.docx`
Finance & Reimbursement	10	`.md`, `.pdf`, `.docx`

Approximately 60% English, 30% Arabic, 10% bilingual (EN+AR). Multilingual embeddings are required.

Demo Target (3-Turn Scenario)

1. **Turn 1 (English, Finance):** *"I traveled to a client meeting last week. How do I submit the reimbursement, and how many days do I have?"* → Router picks Finance, retrieves `FIN-001` and `FIN-002`, cited answer.
2. **Turn 2 (follow-up, exercises memory):** *"And what's the cap on the hotel for a domestic trip?"* → Agent uses session memory, retrieves the cap (1,500 EGP/night), same sources.
3. **Turn 3 (Arabic):** "كم الحد الأقصى لعشاء العملاء لكل شخص؟" → Router picks Finance, retrieves `FIN-009`, returns Arabic answer with RTL rendering.

Tech Stack

- **Required:** Python 3.10+, LangGraph, DeepEval
- **Suggested:** Google Gemini / Groq / Ollama (LLM + embeddings), FAISS / Chroma / Qdrant (vector store), Chainlit / Streamlit / Gradio (UI), multilingual embeddings (`BAAI/bge-m3` or `intfloat/multilingual-e5-large`), structlog / loguru (logging), Pydantic (tool schemas)

- **Cost:** $0 — all free-tier

Deliverables

#	Deliverable	Details
1	**Ingestion pipeline**	Multi-format loader (`.md`, `.pdf`, `.docx`), Arabic-safe extraction, chunking strategy, multilingual embedding, vector store with `domain` / `source_filename` / `language` metadata on every chunk
2	**LangGraph workflow**	Router node, memory loader, domain-filtered retriever, agent with ≥3 typed tools, answer composer, memory writer — typed state schema with `question`, `domain`, `retrieved_chunks`, `tool_calls`, `answer`, `sources`, `session_id`, `run_id`
3	**Session memory**	Full in-session memory keyed by `session_id`; new sessions start with empty memory; reset control in UI
4	**Typed tools (min 3)**	Pydantic input schemas, typed returns; sensible call heuristic; Pydantic validation errors recover without crash
5	**Citations on every answer**	Every answer ends with a "Sources" section of real filenames from `data/docs/`; Arabic answers in Arabic, English answers in English
6	**Chat UI**	Single-command launch; per-turn visible steps (domain, sources, tool calls, cited answer); RTL Arabic rendering; "New chat" reset control
7	**Structured logging**	One JSON line per event tagged with `run_id` → `outputs/run_logs.jsonl`; greppable by `run_id`
8	**DeepEval evaluation script**	`tests/evaluate.py` →

#	Deliverable	Details
		`outputs/eval_report.json`; 5 metrics: Faithfulness, AnswerRelevancy, ContextualPrecision, ContextualRecall, Hallucination; + custom routing-accuracy check
9	**Reset script**	`scripts/reset.sh` (or `.py`) — wipes index and rebuilds from `data/docs/` in one command
10	**Documentation & demo assets**	`docs/architecture.png`, 60–90 s screen recording (`docs/demo.gif` or `.mp4`), 3-page `outputs/final_report.md`, README with 5-command Quick Start

Acceptance Criteria

- Chat UI starts with a single command
- All 3 demo turns run end-to-end with visible routing, retrieval, tool call, and cited answer
- Session memory works — follow-up resolves correctly against prior turn (visible in demo)
- Arabic question returns correct, cited, RTL-rendered answer
- Every answer cites real filenames or says the answer is not in the corpus
- `eval_report.json` includes all 5 DeepEval metrics + routing accuracy
- Zero fabricated policies

Team Division (4 Members)

Member 1 — Data Ingestion & Vector Store

Focus: Getting raw documents into a queryable, metadata-rich vector index.

Tasks:

- Set up the project structure, `requirements.txt`, `.env` / `.env.example`, `.gitignore`, and `src/settings.py` (chunk size, top-k, model names, etc.)
- Implement multi-format document loaders for `.md`, `.pdf`, and `.docx` — ensure Arabic character order is preserved (test with at least one Arabic PDF and one Arabic DOCX)

- Choose and implement a chunking strategy (recursive character splitter, semantic splitter, or paragraph-based); document the choice
- Choose multilingual embeddings that handle both English and Arabic; justify the choice
- Ingest all 30 documents into the vector store with `domain`, `source_filename`, and `language` metadata on every chunk
- Ensure ingestion completes in under 2 minutes on a normal laptop
- Persist the index to disk; load on re-run without rebuilding
- Write `scripts/reset.sh` that wipes the index and rebuilds from scratch
- Explore retrieval quality: run sample queries (including Arabic ones from `eval_cases.jsonl`) in `notebooks/02_retrieval.ipynb` and confirm cross-language retrieval works

Deliverables: `src/ingest.py`, `src/settings.py`, `scripts/reset.sh`, `notebooks/02_retrieval.ipynb`

Member 2 — LangGraph Workflow & Domain Router

Focus: The core agentic graph — routing, retrieval, tool calls, and memory.

Tasks:

- Design and implement the LangGraph state graph in `src/orchestrator.py` with at minimum: router node, memory loader, domain-filtered retriever node, agent node, answer composer, memory writer
- Build the router — classifies questions into `hr` / `it` / `finance`; define a sensible fallback when confidence is low (fan out, default-to-most-likely, or ask user); document the choice
- Define the typed state schema (Pydantic / TypedDict) carrying `question`, `domain`, `retrieved_chunks`, `tool_calls`, `answer`, `sources`, `session_id`, `run_id`
- Implement domain-filtered retrieval (passes domain tag to Member 1's retriever)
- Enforce per-run max-step (8–12) and max-tool-call (3) limits; route to fallback on overflow
- Implement session memory (LangGraph `MemorySaver` or equivalent) keyed by `session_id`
- Build ≥3 typed tools with Pydantic schemas (e.g. policy lookup by ID, reimbursement calculator, leave-types lister); ensure Pydantic validation errors return a feedback message instead of crashing
- Write `config/RAI_Config.yaml` — rules for no fabrication, cite sources, respond in user's language

Deliverables: `src/orchestrator.py`, `src/tools.py`, `src/settings.py` (shared with M1), `config/RAI_Config.yaml`

Member 3 — Chat UI & Observability

Focus: The user-facing experience and full runtime visibility.

Tasks:

- Build the chat UI (Chainlit, Streamlit, Gradio, or FastAPI + HTML — your choice) that launches with a single command
- Per-turn display: chosen domain, retrieved source filenames, any tool calls, and the final cited answer (collapsible panels are fine)
- RTL rendering for Arabic answers — test with at least one Arabic eval question before submission
- "New chat" / "Reset" control that starts a fresh session
- Incremental progress feedback (e.g. "Routing…", "Retrieving…") instead of a frozen screen while the graph runs
- Implement `src/observability.py`: assign a UUID `run_id` at the start of each question; write one JSON line per event (router decision, retrieval hits with filenames, each tool call with input/output, final answer) to `outputs/run_logs.jsonl` using structlog / loguru / stdlib JSON formatter
- Ensure logs are greppable by `run_id` and contain no raw API keys or full document bodies

Deliverables: `app.py` (or `src/app/`), `src/observability.py`, `outputs/run_logs.jsonl` (sample)

Member 4 — Evaluation, Documentation & Demo

Focus: Honest, reproducible measurement and a portfolio-ready presentation.

Tasks:

- Write `tests/evaluate.py`: reads `tests/eval_cases.jsonl`, runs each of the 10 gold cases through the full workflow, and produces `outputs/eval_report.json`
- Implement all 5 required DeepEval metrics: `FaithfulnessMetric`, `AnswerRelevancyMetric`, `ContextualPrecisionMetric`, `ContextualRecallMetric`, `HallucinationMetric` — ensure the judge LLM handles Arabic
- Add a custom routing-accuracy check: compare predicted domain against `expected_domain` per case; report overall accuracy and per-case pass/fail in the same JSON
- `eval_report.json` structure: per-metric averages, pass/fail counts, per-case detail (id, input, predicted domain, expected domain, actual output, expected output, metric scores)
- Write `docs/architecture.png` — LangGraph node layout and end-to-end flow
- Record a 60–90 second demo walkthrough (`docs/demo.gif` or `demo.mp4`) showing the

3-turn reimbursement scenario with memory and one Arabic question

- Write the 3-page `outputs/final_report.md`: architecture, stack choices (and why), LangGraph design, RAG design, routing logic, tools, DeepEval results with comment on each metric, known limitations
- Write the README: project overview, screenshots, 5-command Quick Start, results table, architecture diagram, file tree, known limitations

Deliverables: `tests/evaluate.py`, `outputs/eval_report.json`, `docs/architecture.png`, `docs/demo.gif` / `demo.mp4`, `outputs/final_report.md`, `README.md`